

SAT 求解器的技术演进及其在软件验证中的应用调研

刘鼎琨，武汉大学计算机学院

2026 年 4 月 1 日

摘要

本文系统调研了布尔可满足性（SAT）求解器的技术演进及其在软件验证领域的核心应用。首先，文章回顾了 SAT 问题的基本定义与合取范式（CNF）结构，详细论述了从经典 DPLL 框架向现代冲突驱动子句学习（CDCL）算法的技术跃迁，并探讨了满足多并发验证需求的增量求解技术。在建模与应用层面，本文深入分析了位向量编码、序编码及电路级推理等关键建模技术，并重点剖析了有界模型检测（BMC）的原理及其在红绿灯控制系统中的实例化应用。针对软件安全验证，本文详细阐述了 CBMC 方案如何通过位向量化、多路复用逻辑及循环展开等手段，将 C 程序语义转化为布尔约束。最后，本文追踪了 SAT 研究的前沿趋势，包括图神经网络与强化学习驱动的智能求解、大模型辅助的代码演化、以及面向安全关键领域的 UNSAT 证明校验技术。通过对 Linux 内核配置冲突检查及工业级实例模块化特性的讨论，本文证实了领域专用求解器与硬核协同模式在处理大规模复杂系统验证中的有效性，为未来高性能验证引擎的研发提供了理论支撑。

关键词：SAT；DPLL 算法；CDCL 算法；BMC；CBMC；NeuroSAT；证明校验

1 布尔可满足性问题概述

布尔可满足性问题 (Boolean Satisfiability Problem, SAT) 是判定给定的布尔公式是否存在一组真值指派使其结果为真的计算问题。在计算机科学中, SAT 不仅是第一个被证明为 NP 完全的问题, 也是形式化验证、人工智能规划和现代 EDA 工具的逻辑基石。

1.1 SAT 问题的定义与基本形式

1.1.1 布尔变量与文字 (Literal & Clause)

设 $X = \{x_1, x_2, \dots, x_n\}$ 为一组布尔变量。每个变量的取值空间为 $\{0, 1\}$, 分别代表“假”与“真”。变量 x_i 及其否定 $\neg x_i$ 统称为文字 (Literal)。变量 x_i 称为正文字, $\neg x_i$ 称为负文字。子句 (Clause) 是由文字构成的析取式。

$$C = l_1 \vee l_2 \vee \dots \vee l_k$$

当且仅当子句中至少有一个文字 l_i 为真时, 子句 C 的值为真。

1.1.2 合取范式 (CNF) 的结构与变换

合取范式 (Conjunctive Normal Form, CNF) 是 SAT 问题的标准输入格式。一个 CNF 公式 F 是由一系列子句构成的合取式¹:

$$F = \bigwedge_{j=1}^m C_j = \bigwedge_{j=1}^m \left(\bigvee_{i=1}^{k_j} l_{j,i} \right)$$

在工业应用中, 非 CNF 结构的逻辑电路通常通过 Tseitin 变换转化为 CNF。该变换通过引入辅助变量 a_i 代表中间逻辑门的输出, 避免了直接分配律展开导致的规模指数爆炸¹。例如, 对于逻辑门 $x_3 = x_1 \wedge x_2$, 其对应的等价 CNF 子句集为:

$$(\neg x_1 \vee \neg x_2 \vee x_3) \wedge (x_1 \vee \neg x_3) \wedge (x_2 \vee \neg x_3)$$

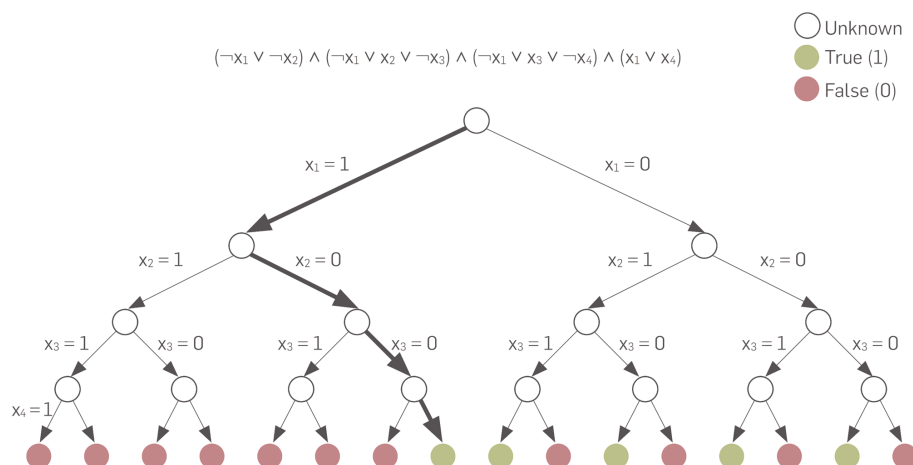
1.2 判定结果: SAT 与 UNSAT

SAT 求解器的核心输出是判定公式 F 在变量集 X 上的满足性。

可满足 (Satisfiable, SAT) 指的是若存在一个真值指派 $\mu : X \rightarrow \{0, 1\}$, 使得 $F(\mu) = 1$, 则称该公式是 SAT 的, μ 称为该公式的一个模型 (Model)。在路径规划中, SAT 意味着存在一条从起点到终点的无碰撞路径; 在硬件设计中, 意味着存在一组输入向量能触发特定的电路行为²。

不可满足 (Unsatisfiable, UNSAT) 指的是若对于任意指派 μ 都有 $F(\mu) = 0$, 则称该公式是 UNSAT 的。在形式化验证中, 若将“系统违背安全规范”的状态建模为 F , 则 UNSAT 结果意味着该非法状态在数学上是不可到达的, 从而证明了系统的安全性²。

全部的可满足指派和不可满足指派构成如图1的搜索空间。

图 1: 一个 CNF 范式的搜索空间样例¹¹

1.3 典型的 SAT 变体及其特点

1.3.1 2-SAT 与 3-SAT 的复杂度阶跃

2-SAT 是指每个子句包含至多两个文字。该问题可以在线性时间 $O(n + m)$ 内求解。其算法原理是构建蕴含图³，其中每个子句 $(a \vee b)$ 被转化为有向边 $(\neg a \rightarrow b)$ 和 $(\neg b \rightarrow a)$ 。满足性的充要条件是：没有任何变量 x_i 及其否定 $\neg x_i$ 位于同一个强连通分量中。

3-SAT 则指的是每个子句包含三个文字。它是标准的 NP 完全问题。Stephen Cook 于 1971 年证明了任何 NP 类问题都可以在多项式时间内归约为 3-SAT⁴。

1.3.2 Max-SAT: 极大化可满足子句数

Max-SAT 是 SAT 的优化版本，旨在寻找使满足子句数量（或权重）最大化的指派。这也引出了部分 Max-SAT (Partial Max-SAT) 问题，即将子句分为硬子句（Hard Clauses，必须满足）和软子句（Soft Clauses，尽可能满足）。其目标函数为：

$$\text{Minimize } \sum_{j \in \text{Soft}} w_j \cdot z_j, \quad \text{s.t. } \forall C_k \in \text{Hard}, C_k(\mu) = 1$$

其中 $z_j = 1$ 表示第 j 个软子句未被满足⁵。

1.3.3 SMT (Satisfiability Modulo Theories)

SMT 是 SAT 在一阶逻辑上的扩展。它允许在布尔骨架上嵌入特定领域的数学理论 T （如线性算术、位向量理论等）。例如，判断下式是否满足：

$$(x > 0 \vee y > 0) \wedge (x + y < 0) \wedge (x, y \in \mathbb{R})$$

SMT 求解器通过 DPLL(T) 架构，结合布尔搜索引擎和专门的理论决策过程来处理非布尔约束⁶。

2 SAT 求解技术的国内外现状调研

从 1962 年的 DPLL 框架到如今的电路级求解器 X-SAT，SAT 技术经历了从理论奠基到工业级应用的巨大跨越。本章通过梳理图2所示的技术演进路线，重点分析 DPLL、CDCL、增量求解、电路求解等核心技术¹¹如何共同支撑起现代 SAT 求解器的卓越性能。

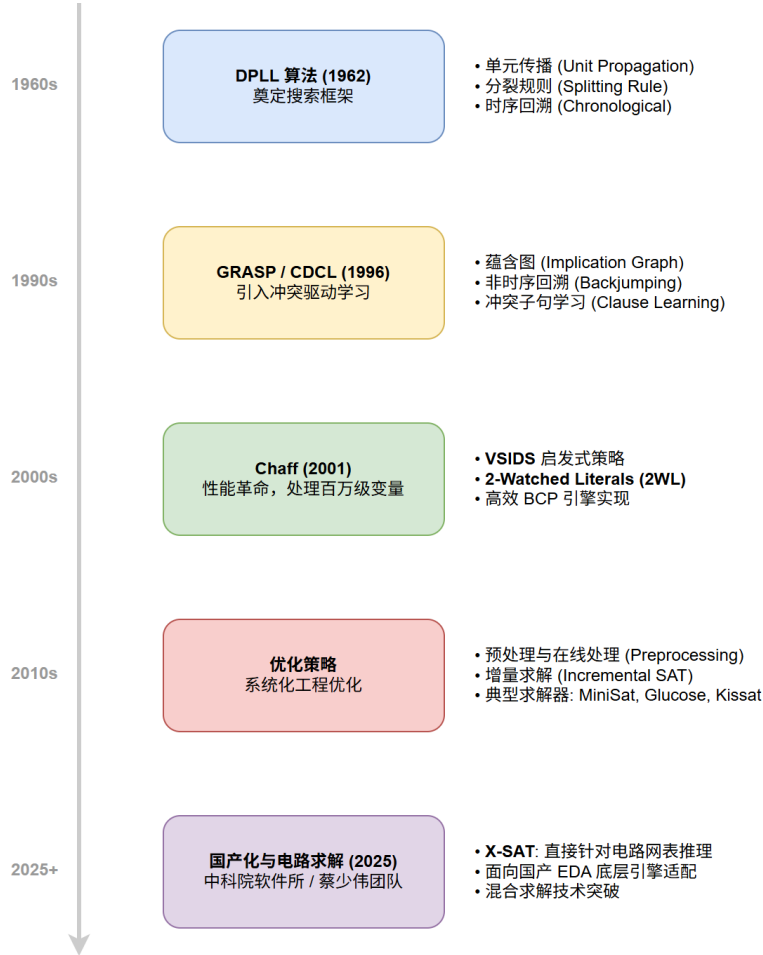


图 2: SAT 求解技术演进路线

2.1 DPLL 算法：基于回溯的深度优先搜索框架

1962 年提出的 DPLL 算法奠定了完备性 SAT 求解器的基础⁷。其核心是一个基于变量赋值和回溯的递归搜索过程。算法的执行逻辑可以表示为递归函数 $DPLL(F, \mu)$ ，其中 F 为公式， μ 为当前的真值指派。其关键步骤有二：

首先考虑单元传播 (Unit Propagation)，若存在单文字子句 (l)，则必须指派 $l = 1$ 。这一过程会引发连锁反应，即 $F \leftarrow F|_{l=1}$ 。若传播导致产生空子句，则可以判定为 UNSAT。

在没有单文字子句的情况下，应遵循分裂规则 (Splitting Rule)，即选择一个未赋值变量 x ，分别探索 $x = 1$ 和 $x = 0$ 两个分支，这便是回溯的递归搜索过程⁷。

2.2 CDCL (Conflict-Driven Clause Learning): 现代求解器的基本逻辑

2.2.1 GRASP 算法: 开山之作

20 世纪 90 年代末, 冲突驱动子句学习 (CDCL) 架构的出现使 SAT 求解规模从几百个变量跃升至数百万个变量。CDCL 引入了“记忆”机制。首先阐述这种设计的是 Marques-Silva 团队的 GRASP 算法。

当产生冲突时, 求解器构建蕴含图 (Implication Graph)。图3中节点代表赋值文字 (记为 $l@d$, 表示文字 l 在第 d 层决策处被赋值), 有向边表示逻辑推导关系⁸。

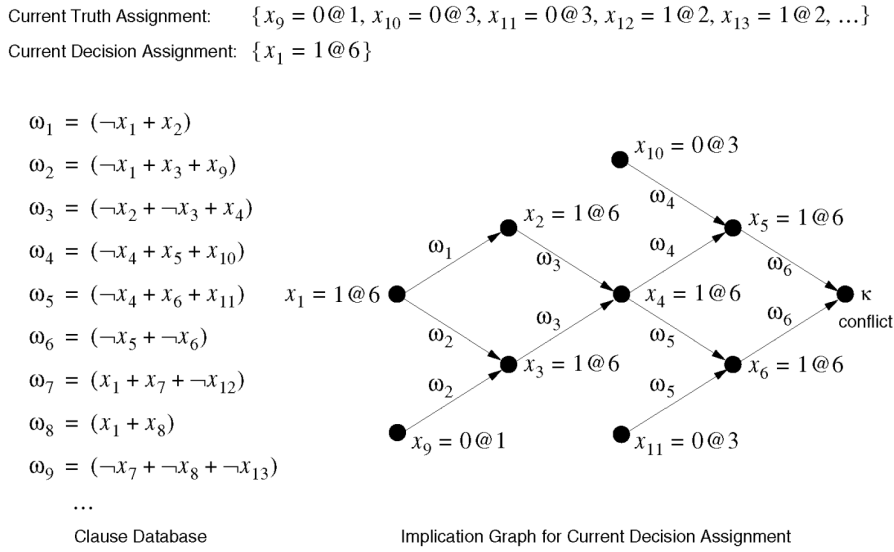


图 3: 子句数据库和部分蕴含图示例⁸

通过对冲突子句和前置子句进行递归的归结运算 (Resolution), 求解器回溯搜索路径, 提取导致冲突的根源约束进行学习, 直至距离冲突最近的单一蕴含点 (Unique Implication Point, UIP), 现代求解器通常提取其作为学习子句。数学上, 可以设冲突子句为 C_{conf} , 前置子句为 C_{ante} , 通过变量 x 的归结公式为:

$$C_{new} = (C_{conf} \setminus \{x\}) \vee (C_{ante} \setminus \{\neg x\})$$

递归至距离冲突最近的单一蕴含点生成的 C_{new} 会被永久存入子句库, 用于剪除未来搜索空间中的重复冲突路径⁸。在图3中的例子, 首先消去变量 x_6 , 将冲突子句 ω_6 与 x_6 的前置子句 ω_5 进行归结运算:

$$\frac{(\neg x_5 \vee \neg x_6) \quad (\neg x_4 \vee x_6 \vee x_{11})}{(\neg x_5 \vee \neg x_4 \vee x_{11})} \text{ (Res on } x_6)$$

得到中间子句 $C_{tmp} = (\neg x_5 \vee \neg x_4 \vee x_{11})$ 。进一步消去变量 x_5 , 将中间子句 C_{tmp} 与 x_5 的前置子句 ω_4 进行归结运算:

$$\frac{(\neg x_5 \vee \neg x_4 \vee x_{11}) \quad (\neg x_4 \vee x_5 \vee x_{10})}{(\neg x_4 \vee \neg x_4 \vee x_{11} \vee x_{10})} \text{ (Res on } x_5)$$

化简得到 $C_{new} = (\neg x_4 \vee x_{10} \vee x_{11})$ 。

与 DPLL 的逐层回溯不同, CDCL 利用学习子句中的信息, 直接跳回到导致冲突的最早决策层 (Asserting Level)。这种“跨层跳跃”称为非时序回溯 (Non-chronological Backjumping), 极大提高了处理具有深逻辑链实例的效率。

2.2.2 Chaff 求解器: 走进工业界

在 2001 年, Chaff 求解器的面世真正让 SAT 求解器走进工业界。其相比于 GRASP 有两大改进:

第一, 采用了启发式变量选择。在搜索过程中, 如何选择下一个分裂变量对效率至关重要。VSIDS (Variable State Independent Decaying Sum) 是现代冲突驱动子句学习 (CDCL) SAT 求解器性能的核心支柱之一, 也是 Chaff 求解器的核心组件⁹。VSIDS 及其变体凭借其独特的“加分-衰减”机制, 是目前为止最有效的策略之一¹⁰。

其核心思想是为每个变量维护一个动态的“活跃度”分数 $S(l)$ 。每当一个变量出现在新学习到的冲突子句中时, 其分数获得一个加性提升 $S(l) \leftarrow S(l) + \Delta$; 同时, 所有变量的分数会周期性地乘以一个小于 1 的衰减因子 (如 0.95), 实现乘性衰减, 即 $S(l) \leftarrow S(l) \cdot \rho$ (其中 $0 < \rho < 1$)。这种机制确保了求解器能持续关注与近期冲突最相关的“活跃”变量, 其本质是一个指数移动平均滤波器, 能有效放大反复引发冲突的变量信号, 滤除噪声⁹。

近年来的研究为 VSIDS 非凡的有效性提供了更深刻的图论解释, 对 VSIDS 算法的理解不局限于选择最近频繁冲突的变量。研究表明, VSIDS 的搜索表现出显著的时空聚焦性⁹。在空间上, 它不成比例地从公式社区结构中的少数几个社区选取变量, 例如在应用类实例上, mVSIDS 的基尼系数高达 0.592, 而随机分支仅为 0.216¹⁰; 在时间上, 它倾向于持续探索近期访问过的社区。这种聚焦性并非随机, VSIDS 无意识地优先选择那些在时序变量关联图中具有高时序中心性的变量¹⁰。

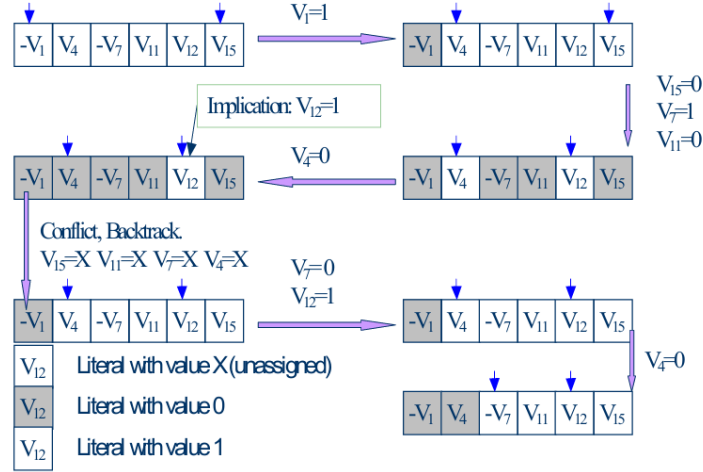
更关键的是, 这些高中心性变量通常是连接不同社区的桥变量。实验证实, 在应用类和组合类实例中, VSIDS 选择进行分支的变量中, 桥变量的比例远超其在变量中的自然比例¹⁰。这暗示 VSIDS 有效地执行了一种“分治”策略: 通过给关键桥变量赋值来分解问题。

第二, 采用了双观测变量 (2-Watched Literals, 2WL)。在 GRASP 等早期求解器中, 为了知道一个子句是否只剩一个未赋值文字, 系统会给每个子句配一个计数器, 每当一个变量 x 被赋值, 程序必须遍历所有包含 $\neg x$ 的子句并更新它们的计数器, 回溯的时候也需要撤销这些操作⁸。2WL 的逻辑是一个子句只要有文字不是 0, 它就不可能成为单位子句, 也不可能产生冲突。案例如图 4, 详细分析可以参照 Chaff 原著⁹。

2.2.3 预处理与在线处理技术: 优化策略

由于工业级 CNF 公式往往包含大量冗余逻辑, 预处理 (Preprocessing) 通过变量消除和子句包容性检查来简化公式¹²。

变量消除基于 Davis-Putnam 规约规则, 通过对变量 x 的所有正负子句进行归结运算并删除原变量, 从而减少变量总数¹³。与此同时, 子句包容性检查识别并删除冗余子句。例如, 若子句 $C_1 \subseteq C_2$, 则 C_2 在逻辑上是多余的, 移除它可以减轻传播引擎的负担。

图 4: 使用 2WL 的 BCP 过程示例⁹

与预处理不同，在线处理 (Inprocessing) 是 CDCL 求解器在搜索过程中交替进行的“在线”优化。其核心在于利用搜索过程中学习到的新子句和当前的赋值状态，重新审视并简化原始公式¹⁴。例如，可以通过分析蕴含关系，识别在当前约束下始终取相同值的变量并进行合并。

这些技术在求解器发展史上具有里程碑意义。MiniSat 2.0 通过集成 SatELite 预处理器的核心算法（如 ‘SimpSolver’ 类），首次在工业赛道上证明了变量消除的巨大威力。而现代顶尖求解器如 CaDiCaL、Kissat 和 Glucose 则更进一步，将这些逻辑深度嵌入搜索循环的间隙中，使求解器具备了在搜索过程中“动态重构”问题的能力¹¹。

2.3 增量 SAT 求解 (Incremental SAT)

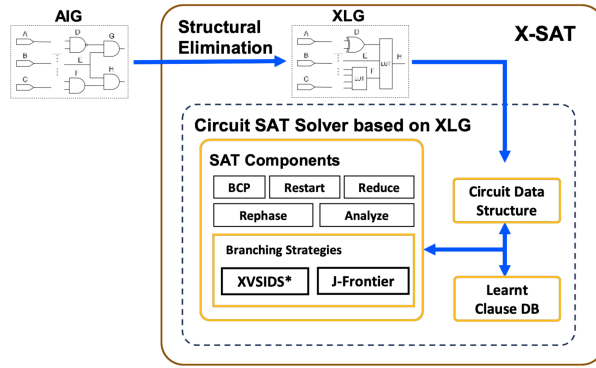
在诸如有限模型检测 (BMC)、自动化规划和软硬件形式化验证等工业应用中，求解任务往往表现为一系列高度相似的逻辑公式序列。增量 SAT 求解 (Incremental SAT Solving) 技术允许求解器在处理该序列时，保留并复用之前搜索过程中积累的状态信息，从而避免从零开始的冗余计算¹⁵。具体代数实现这里不做展开。

自 2014 年 SAT Competition 引入以来，IPASIR (Re-entrant Incremental SAT API) 已成为现代求解器的通用标准。它规定了 ‘add()’（添加永久子句）、‘assume()’（设置单次假设）和 ‘solve()’（执行求解）的三大核心操作¹⁶。

2.4 最新国内外研究现状

在国际研究视野中，年度 SAT Competition 竞赛始终是衡量求解器性能与算法创新的风向标。其中，由 Armin Biere 团队研发的 Kissat 求解器凭借其在不可满足性 (UNSAT) 证明生成及布尔约束传播 (BCP) 优化方面的卓越表现，长期处于全球领先地位¹⁷。与此同时，国内在约束求解领域的研究正步入爆发期。以中国科学院软件研究所基础软件与系统重点实验室为代表，蔡少伟研究员带领的团队在局部搜索与电路 SAT 方向取得了显著突破。

在 EDA 领域顶级会议 DAC 2025 上，该团队正式提出了自主研发的电路求解器 X-SAT。该求解器摒弃了传统的 CNF 编码方式，转而直接以门级网表作为推理单元，如图5。通过创新性

图 5: X-SAT 框架图¹⁸

的 XLG 门级网表结构（仅包含 XOR 门与 LUT 门）大幅缩减逻辑规模。实验数据表明，X-SAT 在处理乘加混合电路等困难实例时，性能相较于国际顶尖 CNF 求解器 Kissat 提升了 35.94%¹⁸。

3 SAT 的典型应用场景与案例剖析

3.1 建模方法：从声明式编程到布尔编码

SAT 求解的本质是一种声明式编程，用户通过逻辑约束描述“解应该满足什么条件”，而非规定“如何去寻找解”¹⁴。由于现代求解器（如 Kissat、Z3）对 CNF 或 SMT-LIB 格式有严格要求，建模质量直接决定了求解效率。以下介绍三种将现实世界中的复杂问题（如软件逻辑、电路设计、资源调度）转化为 SAT 求解器能够理解的“布尔语言”的核心技术。

第一，位向量编码 (Bit-blasting)。在处理算术逻辑（如软件中的 $x + y = z$ ）时，求解器将 w 位整数映射为 w 个布尔变量，并利用门级逻辑（全加器、乘法器电路）构建对应的 CNF 子句，适用于对软件溢出、位运算的还原。这是 SMT 求解器（如 Z3、Bitwuzla）处理位向量理论的核心¹⁹。

第二，序编码 (Order Encoding)。对于线性约束 $x \leq k$ ，引入布尔变量 $b_{x \leq i}$ 表示 x 是否小于等于 i 。这种编码方式在处理资源调度和约束优化问题时，能比传统的二进制编码提供更强的单位传播能力，能大幅减少搜索分支²⁰。

第三，电路级推理 (Circuit-level Reasoning)。如 X-SAT 所采用的策略，直接保留门级网表 (XOR、LUT) 信息，避免了 Tseitin 变换引入大量辅助变量导致原始语义丢失的问题¹⁸。

3.2 重点案例：形式化验证中的有界模型检测 (BMC)

有界模型检测 (Bounded Model Checking, BMC) 是 SAT 技术最成功的工业应用之一，广泛应用于 Intel、AMD 及国产处理器（如 RISC-V 内核）的逻辑验证。

3.2.1 建模原理：状态迁移的布尔展开

BMC 将一个有限状态系统 M 及其待验证属性 P 的验证问题转化为 SAT 判定问题。其核心逻辑是将系统的状态迁移关系 T 在有限步长 k 内进行循环展开。

设 $I(s_0)$ 为初始状态公式， $T(s_i, s_{i+1})$ 为状态从 i 到 $i+1$ 的迁移关系， $P(s_i)$ 为在状态 s_i 下应满足的安全属性 (Invariant)。判定在 k 步内是否存在反例的布尔公式为²¹：

$$F_k = I(s_0) \wedge \bigwedge_{i=0}^{k-1} T(s_i, s_{i+1}) \wedge \bigvee_{i=0}^k \neg P(s_i)$$

若 F_k 为 SAT，求解器给出的赋值即为一个反例轨迹，指示了触发故障的输入序列；若 F_k 为 UNSAT，则证明系统在 k 步范围内是绝对安全的²¹。

3.2.2 案例分析：红绿灯控制系统

假设红绿灯只有红 (R)、黄 (Y)、绿 (G) 三个灯，希望验证一个程序是否会出现错误的亮灯顺序。在 BMC 中，因为时间是离散的，我们定义布尔变量：第 i 秒的状态 s_i 由三个变量 $\{R_i, Y_i, G_i\}$ 决定。如果 $R_i = 1$ ，表示第 i 秒红灯亮。

编码初始状态定义为红灯亮，即 $I(s_0) = R_0 \wedge \neg Y_0 \wedge \neg G_0$ 。其后逻辑建模中，要描述红绿灯转换的编码迁移关系 $T(s_i, s_{i+1})$ ，即红 $\rightarrow$ 绿 $\rightarrow$ 黄 $\rightarrow$ 红。可以用逻辑蕴含来表达转换：红变绿为 $R_i \rightarrow G_{i+1}$ （等价于 $\neg R_i \vee G_{i+1}$ ），绿变黄为 $G_i \rightarrow Y_{i+1}$ （等价于 $\neg G_i \vee Y_{i+1}$ ），黄变红为 $Y_i \rightarrow R_{i+1}$ （等价于 $\neg Y_i \vee R_{i+1}$ ）。为了严谨，还得加上**互斥约束**（每一时刻只能亮一个灯）：

$$Unique_i = (R_i \vee Y_i \vee G_i) \wedge (\neg R_i \vee \neg Y_i) \wedge (\neg Y_i \vee \neg G_i) \wedge (\neg R_i \vee \neg G_i)$$

则最终的迁移关系 $Transition(s_i, s_{i+1})$ 就是这些子句的合集：

$$Transition(s_i, s_{i+1}) = T(s_i, s_{i+1}) \wedge Unique_i \wedge Unique_{i+1}$$

假设我们的安全要求是：“绝对不能出现绿灯和黄灯同时亮”，那么编码安全属性 $P(s_i) = \neg(G_i \wedge Y_i)$ 。要找的是反例，所以要编码其否定形式： $\neg P(s_i) = G_i \wedge Y_i$ 。完整的 BMC 公式展开 ($k=1$ 步) 则为：

$$F_1 = \underbrace{(R_0 \wedge \neg Y_0 \wedge \neg G_0)}_{I(s_0)} \wedge \underbrace{Transition(s_0, s_1)}_{T(s_0, s_1)} \wedge \underbrace{(G_0 \wedge Y_0 \vee G_1 \wedge Y_1)}_{\neg P(s_0) \vee \neg P(s_1)}$$

3.2.3 代码验证方案：CBMC

CBMC 的核心任务是将 C 程序的执行语义转化为布尔公式 F_k ，可以理解为一种特殊的编码方式。以下展示了部分不同程序结构在 Bit-blasting 过程中的转换逻辑。

对于基本的算术运算，CBMC 将变量视为位向量，将操作符转换为等价的数字电路子句。以下为例：

Listing 1: 算术运算的位向量编码

```
unsigned char a, b, c;
c = a + b;
```

每一个变量被映射为 8 个布尔变量（如 $a_7 \dots a_0$ ）。赋值语句 $c = a + b$ 被转换为一个 8 位全加器电路的约束集合： $c_i \leftrightarrow (a_i \oplus b_i \oplus carry_{i-1})$ ，即 $carry_i \leftrightarrow (a_i \wedge b_i) \vee (carry_{i-1} \wedge (a_i \oplus b_i))$ 。

对于条件分支，CBMC 使用多路复用逻辑来消除代码中的条件分支。以下为例：

Listing 2: 条件分支的多路复用编码

```
if (cond) x = a;
else x = b;
```

引入一个中间变量 x_{new} ，通过类似于硬件选择器的逻辑进行编码： $x_{new} \leftrightarrow (cond \wedge (x_{new} = a)) \vee (\neg cond \wedge (x_{new} = b))$ 。这确保了无论 $cond$ 最终被求解器指派为 0 还是 1， x_{new} 都能获得正确的约束。

对于循环结构，CBMC 根据设定的边界 k 进行物理展开，将其转化为一长串的条件赋值。以下为例：

Listing 3: 循环结构的物理展开

```
for (int i=0; i<2; i++) { sum += i; }
```

当 $k = 2$ 时，逻辑被展开为：

Listing 4: 循环展开后的等价逻辑

```
// 第一步
if (0 < 2) { sum_1 = sum_0 + 0; }
// 第二步
if (1 < 2) { sum_2 = sum_1 + 1; }
// 边界检查 (Assertion)
assert(!(2 < 2));
```

每一行 `if` 再按照上述“多路复用”方式转为布尔子句。

除了上述常见的结构外，还可以定义数组、指针等多种结构的 CBMC 转换，程序可以通过合取运算将多步代码合并到同一个 CNF 范式中，进而执行 BMC 算法。

3.2.4 工业级实现：从 BMC 到 IC3

虽然 BMC 在发现浅层 Bug 上极其高效，但它是“不完备”的。这是容易理解的。如上述代码中需要假定 $k = 2$ ，而一旦真实执行次数超过范围就无法证明安全。现代工具如 Synopsys 的 **VC Formal** 和 Cadence 的 **JasperGold** 结合了 **IC3/PDR (Property Directed Reachability)** 算法。IC3 通过增量式学习归纳性子句，能够在不进行显式 k 步展开的情况下，证明无限步长下的系统安全性²⁰。

3.3 现实系统中的关联与方案有效性讨论

3.3.1 现实系统关联：Linux 内核配置冲突检查

在我们身边的复杂系统中，Linux 内核的配置模型（Kconfig）就是一个巨大的类 SAT 问题。Linux 内核拥有超过 10,000 个可配置选项（Features），它们之间存在复杂的相互依赖和排他约束。

Undertaker 是一个利用 SAT 技术检查 Linux 配置一致性的工具。它将 Kconfig 文件的逻辑提取为命题公式，例如：

$$\text{CONFIG_A} \implies (\text{CONFIG_B} \vee \neg \text{CONFIG_C})$$

通过求解，Undertaker 成功识别了 Linux 内核中 1,776 个配置缺陷（包括在任何配置下都无法编译到的“死代码”），这些缺陷中许多已被证实为潜在的 Bug 并合并入主线分支²¹。

3.3.2 方案有效性：为什么能处理百万级变量？

根据计算复杂性理论，SAT 是 NP 完全的，最坏情况下时间复杂度为指数级。然而，现代求解器能够高效处理工业级实例，其核心原因在于工业实例的非随机性。研究表明，工业级 SAT 实例具有极高的模块化（Modularity, Q 值），通常在 0.7 以上，而随机生成的实例 Q 值接近 0。且现代 CDCL 求解器中的 VSIDS 启发式无意识地优先选择那些连接不同逻辑模块的桥变量（Bridge Variables）进行赋值。这实际上在搜索过程中不断将复杂的大问题“切分”为独立的小子问题，从而利用分治思想规避了指数爆炸¹⁰。

4 SAT 研究的热点与未来趋势

4.1 SAT 与机器学习的深度结合

随着深度学习技术的成熟，研究者开始利用图神经网络（GNN）或强化学习（RL）来优化求解器的核心决策环节。

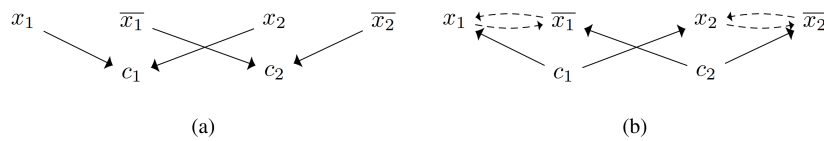


图 6: “变量-子句”二部图学习流程示例²²

神经求解器 (NeuroSAT) 方向，通过将 CNF 公式建模为“变量-子句”二部图。其中，利用 GNN 进行文字嵌入。其核心消息传递机制可表示为：

$$h_v^{(k)} = \text{Update} \left(h_v^{(k-1)}, \sum_{c \in N(v)} \text{Message}(h_c^{(k-1)}) \right)$$

其中 $h_v^{(k)}$ 代表变量 v 在第 k 层的隐藏状态。如图6, 变量状态更新子句状态, 子句状态和变量的逆会共同作用于更新新的变量状态, 这样可以确保在结果层面变量及其逆只有一个为真。尽管神经求解器目前在处理工业大规模实例的完备性上仍有差距, 但在寻找随机难题的满足赋值方面表现卓越²²。

强化学习方向, 利用 RL 辅助 CDCL 在求解过程中动态调整分支策略或重启频率。例如, LangSAT 框架通过 RL 优化启发式选择, 使求解器能更好地适应自然语言转化而来的逻辑约束²³。

大模型辅助求解器设计 (LLM for Solvers) 方向, 前沿趋势是利用大规模语言模型驱动求解器代码的自我演化。SATLUTION 项目通过“生成-测试-反馈”闭环, 自动优化 C++ 核心算法, 其进化出的求解器在 2025 年 SAT 竞赛中击败了多个人工设计的冠军模型²⁴。

4.2 可解释求解与 UNSAT 证明校验

在航空航天、自动驾驶等安全关键领域, 用户不满足求解器只给出 UNSAT 的结论, 因为可能是代码绝对正确, 也可能是代码中出现漏掉应有的问题。因此需要生成 UNSAT 的“绝对证据”。

求解器需输出 DRAT、LRAT 或 FRAT 格式的不可满足证明。DRAT 基于反向单元传播 (RUP) 原理²⁵。对于新加入的学习子句 C , 其合法性的充要条件是:

$$F \wedge \neg C \vdash_{UP} \perp$$

即原始公式与子句否定的合取在单元传播下必须导致矛盾²⁵, 这个思路类似于反证法。LRAT (Linear RAT) 在证明中显式地指出了“利用了哪些子句触发了单元传播”。这省去了校验器搜索的时间, 实现了线性的时间校验²⁵。

另一个前沿方向是“边求解边校验”, 即在线校验 (On-the-fly Checking)。Michaelson 等人提出了一种分布式 LRAT 校验方案。这种方案不再生成巨大的中间文件, 而是在内存中实时校验。一旦某个学习子句被推导出来, 校验线程立即对其进行 LRAT 检查。在分布式或多线程环境下, 利用轻量级加密签名标识已校验子句。只有带“可信签章”的子句才能在不同求解线程间共享, 防止故障在分布式节点间扩散, 将校验开销压缩至 42% 以内²⁶。

4.3 领域专用求解器与硬件加速

通用求解器在面对特定结构 (如密集 XOR 链或大加法器电路) 时往往效率骤降。领域专用化已成为提升性能的主阵地。如中科院蔡少伟团队研发的 X-SAT。它直接在 XLG 网表上执行 CDCL 搜索, 引入了变元搜索启发式 XVSIDS, 获得了很大的性能提升¹⁸。同时, Synopsys 推出的 HAPS-200 和 ZeBu-200 系统通过 FPGA 硬件加速 SAT 逻辑的并行传播。这种“软硬协同”的模式使验证周期缩短了几个数量级, 是现代 SoC 设计的必备工具²⁷。

5 结论

布尔可满足性问题（SAT）已经从一个纯粹的计算复杂性理论模型，演变为现代数字文明中不可或缺的逻辑算力引擎。从 20 世纪 60 年代简单的 DPLL 递归，到现代具备“记忆”与“学习”能力的 CDCL 架构，再到 2025 年崭露头角的 AI 自进化求解器，SAT 技术的每一次飞跃都深刻影响着软件工程、半导体设计及网络安全。

本文通过调研发现，现代求解器之所以能处理百万级变量，其核心在于成功挖掘了工业实例的“社区结构”，利用 VSIDS 等启发式策略无意识地执行了逻辑上的分治策略。随着国产 EDA 工具自主化的提速，如 X-SAT 等电路级专用求解器的突破，标志着我国在这一核心计算领域已进入全球第一梯队。未来，SAT 技术将朝着强智能化、全可信化、深领域化的方向继续演进，成为支撑复杂工程约束求解与智能系统验证的不可替代之石。

参考文献

- [1] Biere, A., van Maaren, H., & Walsh, T. (Eds.). (2009). *Handbook of satisfiability*. SAGE Publications Limited.
- [2] Heule, M. J., & Biere, A. (2021). *Proof Systems*. In Handbook of Satisfiability.
- [3] Aspvall, B., Plass, M. F., & Tarjan, R. E. (1979). A linear-time algorithm for testing the truth of certain quantified boolean formulas. *Information processing letters*, 8(3), 121-123.
- [4] Karp, R. M. (2009). Reducibility among combinatorial problems. In *50 Years of Integer Programming 1958-2008: from the Early Years to the State-of-the-Art* (pp. 219-241). Berlin, Heidelberg: Springer Berlin Heidelberg.
- [5] Li, C. M., & Manyà, F. (2021). *Max-SAT*. In: Biere, A., et al. (Eds.), *Handbook of Satisfiability (2nd edition)*, Chapter 23, pp. 895-937. IOS Press.
- [6] Barrett, C., et al. (2021). *Satisfiability Modulo Theories*. Handbook of Satisfiability.
- [7] Davis, M., Logemann, G., & Loveland, D. (1962). A machine program for theorem-proving. *Communications of the ACM*, 5(7), 394-397.
- [8] Marques-Silva, J. P., & Sakallah, K. A. (2002). GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on computers*, 48(5), 506-521.
- [9] Moskewicz, M. W., Madigan, C. F., Zhao, Y., Zhang, L., & Malik, S. (2001, June). Chaff: Engineering an efficient SAT solver. In *Proceedings of the 38th annual Design Automation Conference* (pp. 530-535).
- [10] Liang, J. H., Ganesh, V., Zulkoski, E., Zaman, A., & Czarnecki, K. (2015, November). Understanding VSIDS branching heuristics in conflict-driven clause-learning SAT solvers. In *Haifa Verification Conference* (pp. 225-241). Cham: Springer International Publishing.
- [11] Malik, S., & Zhang, L. (2009). Boolean satisfiability from theoretical hardness to practical success. *Communications of the ACM*, 52(8), 76-82.
- [12] Eén, N., & Biere, A. (2005, June). Effective preprocessing in SAT through variable and clause elimination. In *International conference on theory and applications of satisfiability testing* (pp. 61-75). Berlin, Heidelberg: Springer Berlin Heidelberg.
- [13] Chatalic, P., & Simon, L. (2000, June). ZRES: The old Davis–Putnam procedure meets ZBDD. In *International Conference on Automated Deduction* (pp. 449-454). Berlin, Heidelberg: Springer Berlin Heidelberg.

- [14] Järvisalo, M., Heule, M. J., & Biere, A. (2012, June). Inprocessing rules. In *International Joint Conference on Automated Reasoning* (pp. 355-370). Berlin, Heidelberg: Springer Berlin Heidelberg.
- [15] Eén, N., & Sörensson, N. (2003, May). An extensible SAT-solver. In *International conference on theory and applications of satisfiability testing* (pp. 502-518). Berlin, Heidelberg: Springer Berlin Heidelberg.
- [16] Fazekas, K., Biere, A., & Scholl, C. (2019, June). Incremental inprocessing in SAT solving. In *International Conference on Theory and Applications of Satisfiability Testing* (pp. 136-154). Cham: Springer International Publishing.
- [17] Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., & Pollitt, F. (2024). CaDiCaL, Gimsatul, IsaSAT and Kissat entering the SAT competition 2024. *Proc. of SAT Competition*, 8.
- [18] Qian, Y., Chen, Z., Zhang, X., & Cai, S. (2025, June). X-SAT: An Efficient Circuit-Based SAT Solver. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)* (pp. 1-7). IEEE.
- [19] Prasad, M. R., Biere, A., & Gupta, A. (2005). A survey of recent advances in SAT-based formal verification. *International Journal on Software Tools for Technology Transfer*, 7(2), 156-173.
- [20] Hu, G., Tang, J., Yu, C., Zhang, W., & Zhang, H. (2024, January). Deepic3: Guiding ic3 algorithms by graph neural network clause prediction. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)* (pp. 262-268). IEEE.
- [21] Tartler, R., Sincero, J., Dietrich, C., Schröder-Preikschat, W., & Lohmann, D. (2012). Revealing and repairing configuration inconsistencies in large-scale system software. *International Journal on Software Tools for Technology Transfer*, 14(5), 531-551.
- [22] Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., & Dill, D. L. (2018). Learning a SAT solver from single-bit supervision. *arXiv preprint arXiv:1802.03685*.
- [23] Pan, M., Walter, M., Kodakandla, D., & Farooque, M. (2025). LangSAT: A Novel Framework Combining NLP and Reinforcement Learning for SAT Solving. *arXiv preprint arXiv:2512.04374*.
- [24] Yu, C., Liang, R., Ho, C. T., & Ren, H. (2025). Autonomous code evolution meets np-completeness. *arXiv preprint arXiv:2509.07367*.
- [25] Wetzler, N., Heule, M. J., & Hunt Jr, W. A. (2014, July). DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *International Conference on Theory and*

- Applications of Satisfiability Testing* (pp. 422-429). Cham: Springer International Publishing.
- [26] Schreiber, D. (2024). Trusted scalable SAT solving with on-the-fly LRAT checking. In *27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024)* (pp. 25-1). Schloss Dagstuhl–Leibniz-Zentrum für Informatik.
- [27] Sutisna, N., Lanante, L., Nagao, Y., Kurosaki, M., & Ochi, H. (2014, November). Live demonstration: hardware-software co-verification for very large scale SoC using synopsys HAPS platform. In *2014 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)* (pp. 171-172). IEEE.